Dofbot_2026 环境安装说明

1. 准备

- 将项目放在**不含中文、空格或特殊字符**的路径下，例如 Windows `D:\Dofbot_2026`、Ubuntu/macOS `~/Dofbot_2026`。
- 下文命令均在**项目根目录**（包含 `requirements.txt` 的目录）执行。

```
Dofbot_2026/
├─ dataset/  models/  results/  utils_learn/
├─ requirements.txt
└─ dofbot.py  dofbot_demo.py  dh_kine_student.py  main_student.py  dofbot_learn_student.py
```

2. 安装 Python 3.10（二选一）

两种方案**任选其一**，都使用 Python 3.10：

方案	适用场景	说明
A 官方 Python	习惯直接使用系统 Python	见 2.1
B Anaconda（推荐）	希望隔离依赖	见 2.2

后续命令按下表选择解释器（方案 B 需先 `conda activate Dofbot_2026`）：

系统	方案 A	方案 B
Windows	`python`（未加入 PATH 时用 `py -3.10`）	激活后 `python`
Ubuntu / macOS	`python3.10`	激活后 `python`

2.1 方案 A：官方 Python 3.10 + pip（推荐）

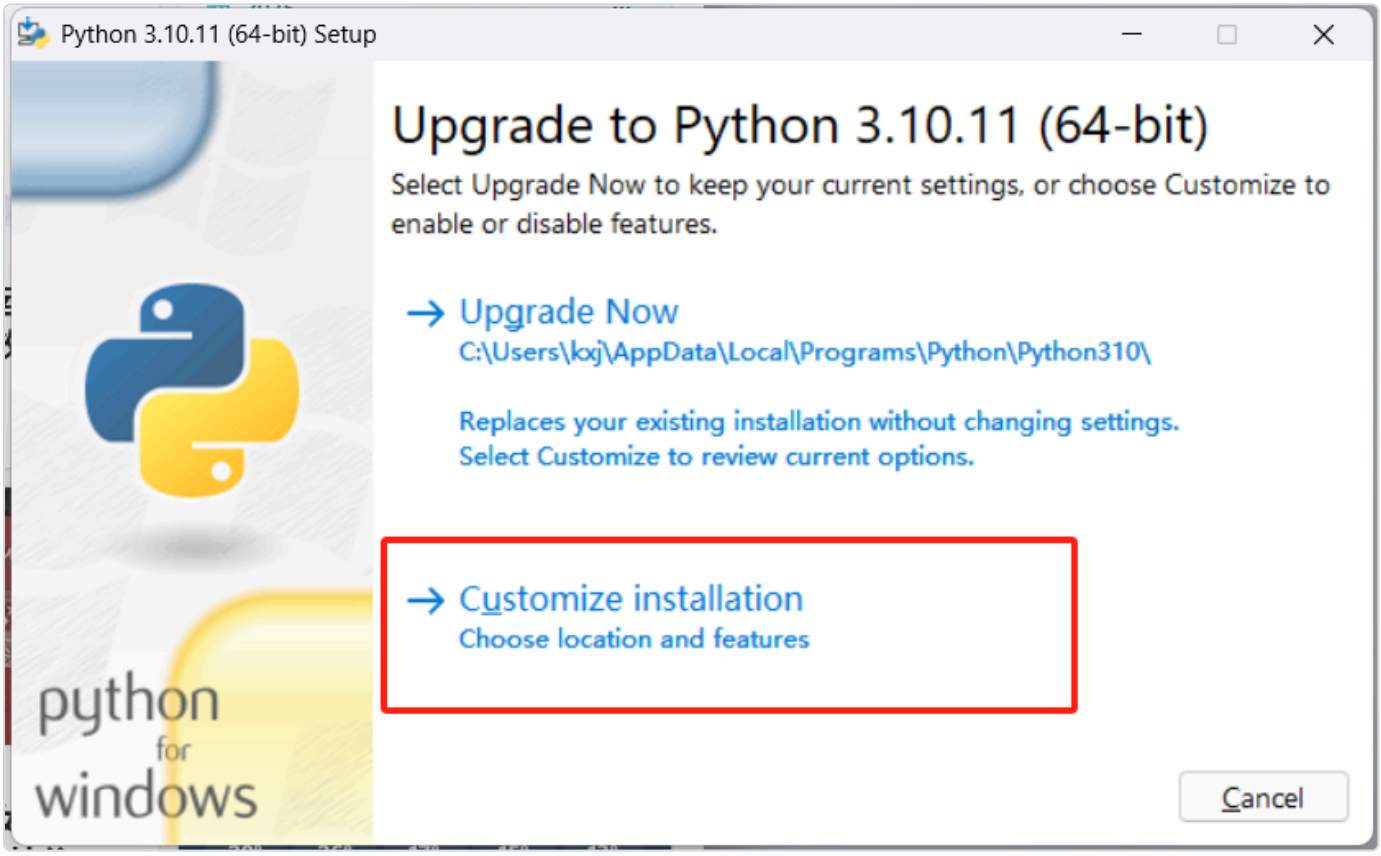
Windows

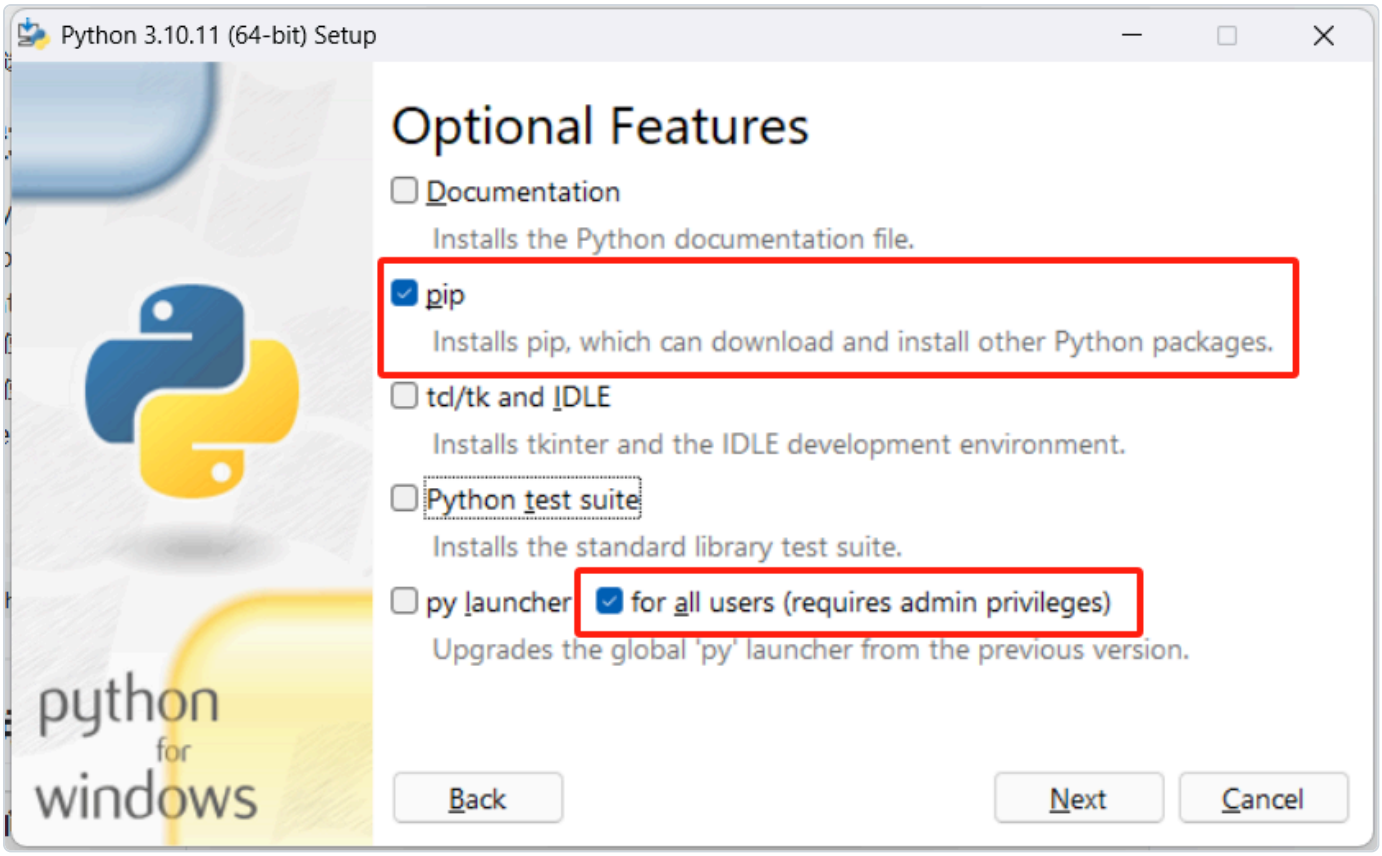
1. **下载**：Python 3.10.11（64-bit）；其他版本见 Python 官网 的 Windows installer（按 CPU 位数选择）。
2. **安装选项**：

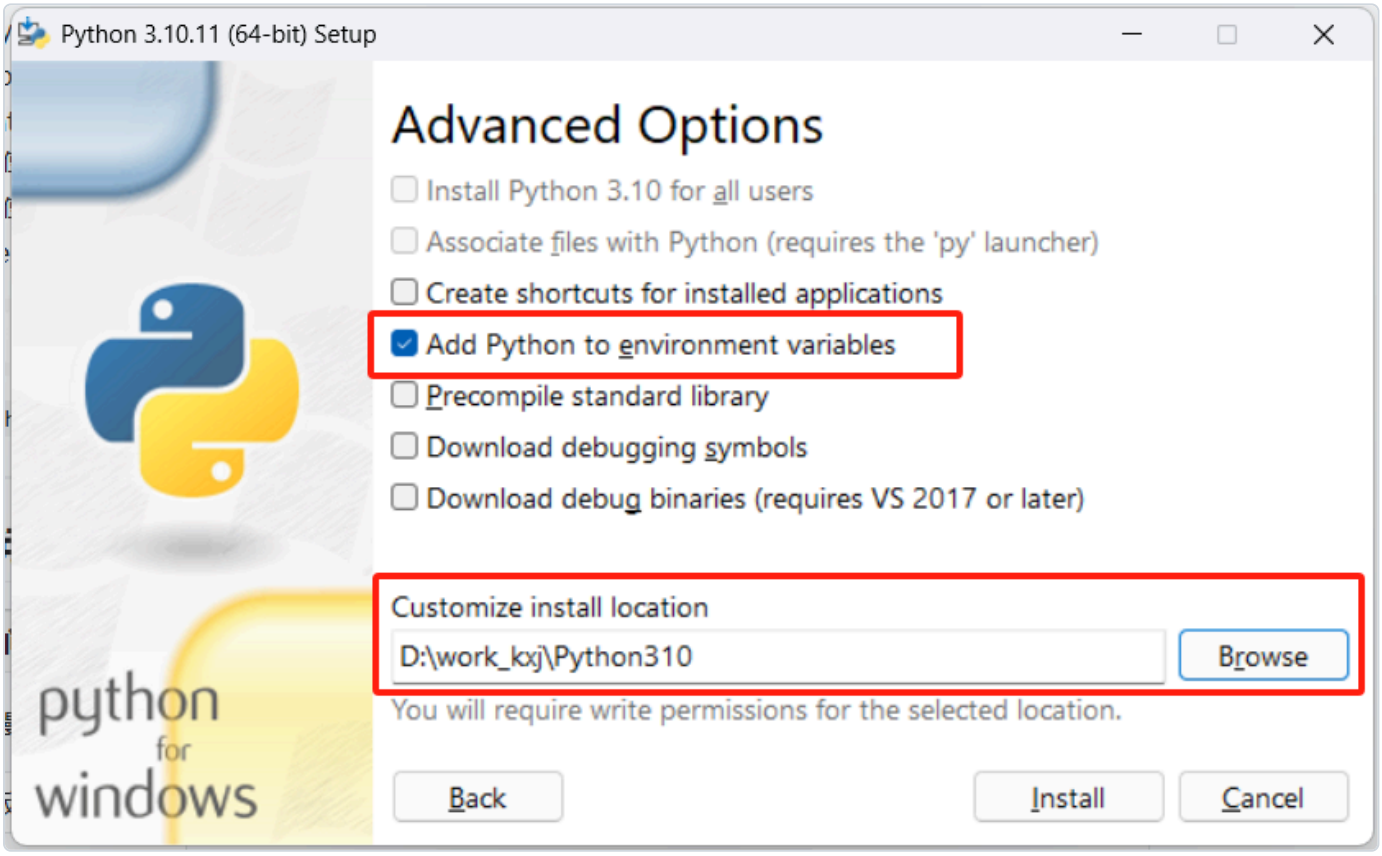
- 勾选 **Add python.exe to PATH**
- 选择 **Customize installation**，勾选 **Install for all users**、**Precompile standard library**
- 安装路径建议：`D:\Python310`

Files

Version	Operating System	Description	MD5 Sum	File Size	Sigstore	SBOM	GPG
Gzipped source tarball	Source release		5d0c0e4c6a022a87165a9addcd869109	25.8 MB	.sigstore	SPDX	SIG
XZ compressed source tarball	Source release		c6c933c1a0db52597cb45a7910490f93	19.5 MB	.sigstore	SPDX	SIG
macOS 64-bit universal2 installer	macOS	for macOS 10.13 and later	82711848a795f6d7b25e81844d5a9a3f	43.3 MB	.sigstore		SIG
Windows installer (64-bit)	Windows	Recommended	b51e0889be50c55fbdd809f4ad587120	25.3 MB	.sigstore	SPDX	SIG
Windows installer (32-bit)	Windows		5d5452249401822cb3ad1bce7105d5fd	24.1 MB	.sigstore	SPDX	SIG
Windows installer (ARM64)	Windows	Experimental	19bdd2de8a7ccb6f1115f85bc54c1764	24.6 MB	.sigstore	SPDX	SIG
Windows embeddable package (64-bit)	Windows		4c0a5a44d4ca1d0bc76fe08ea8b76adc	10.6 MB	.sigstore	SPDX	SIG
Windows embeddable package (32-bit)	Windows		21a051ecac4a9a25fab169793ecb6e56	9.4 MB	.sigstore	SPDX	SIG
Windows embeddable package (ARM64)	Windows		6fc899d8dbd46dd2b585a038f7cf68a4	9.8 MB	.sigstore	SPDX	SIG







3. **验证**：按 `Win + R` 输入 `cmd`，执行：

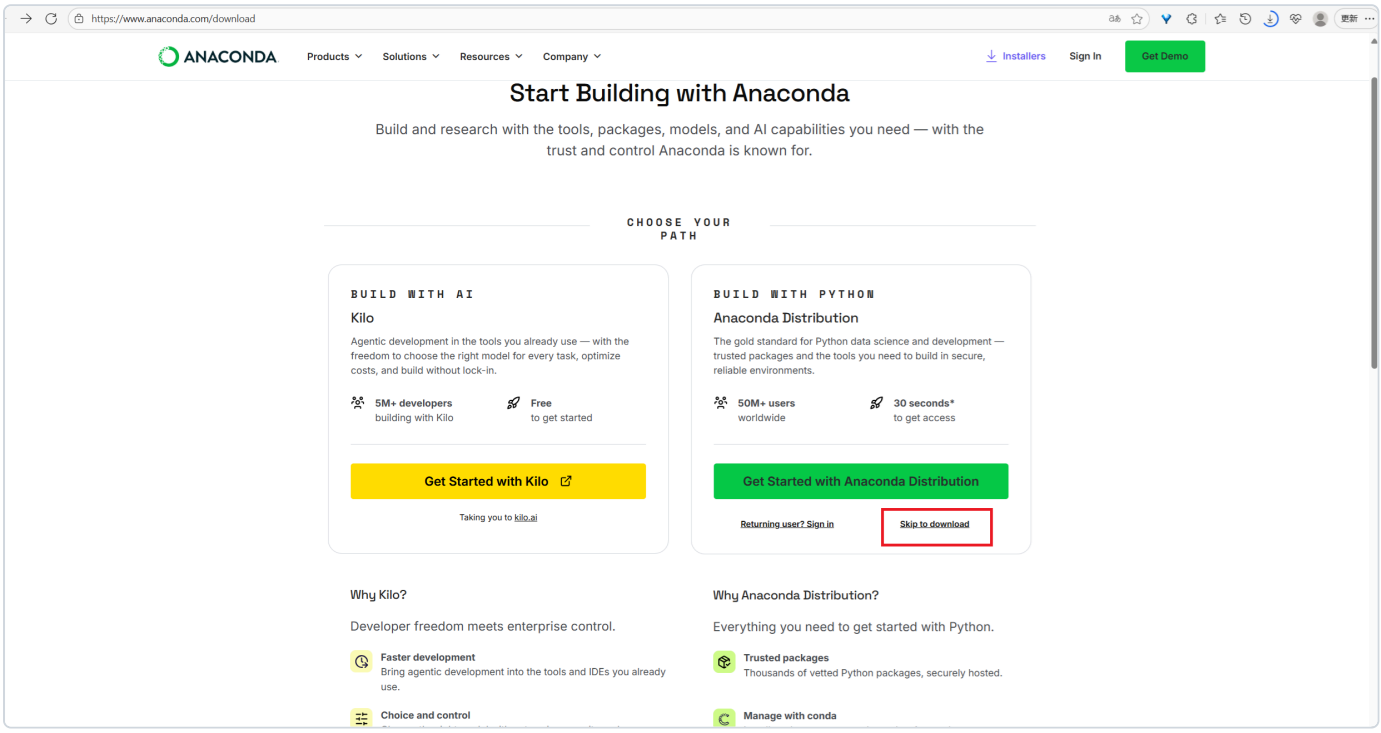
```bat
python -V # 应输出 Python 3.10.11
pip -V # 确认 pip 可用
```

> 若电脑已装过其他版本的 Python，建议先卸载或清理环境变量，避免冲突；否则改用方案 B（Anaconda）。

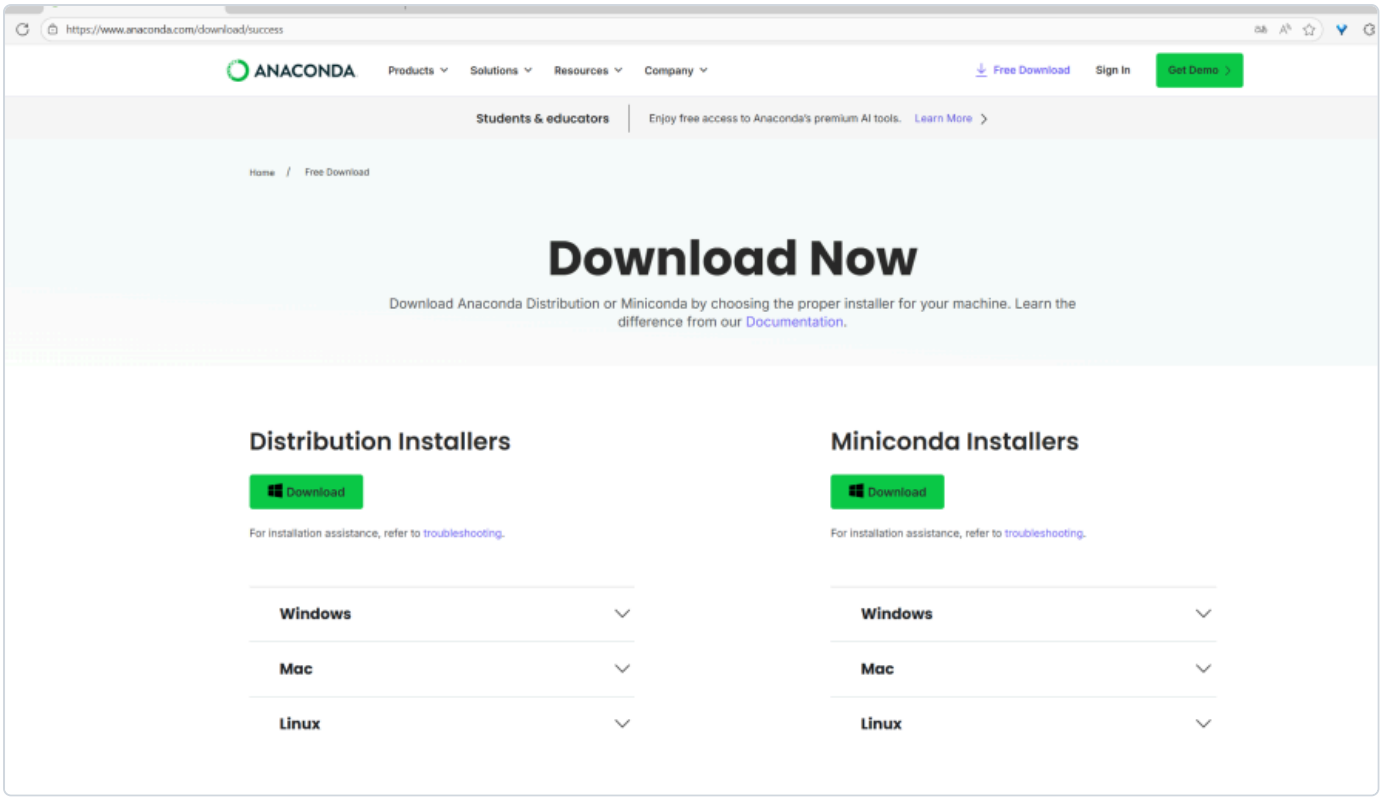
Ubuntu / macOS：Ubuntu 执行 `sudo apt install python3.10 python3-pip`；macOS 用官网 universal2 安装包。系统源无 3.10 时改用方案 B。

2.2 方案 B：安装 Anaconda（推荐）

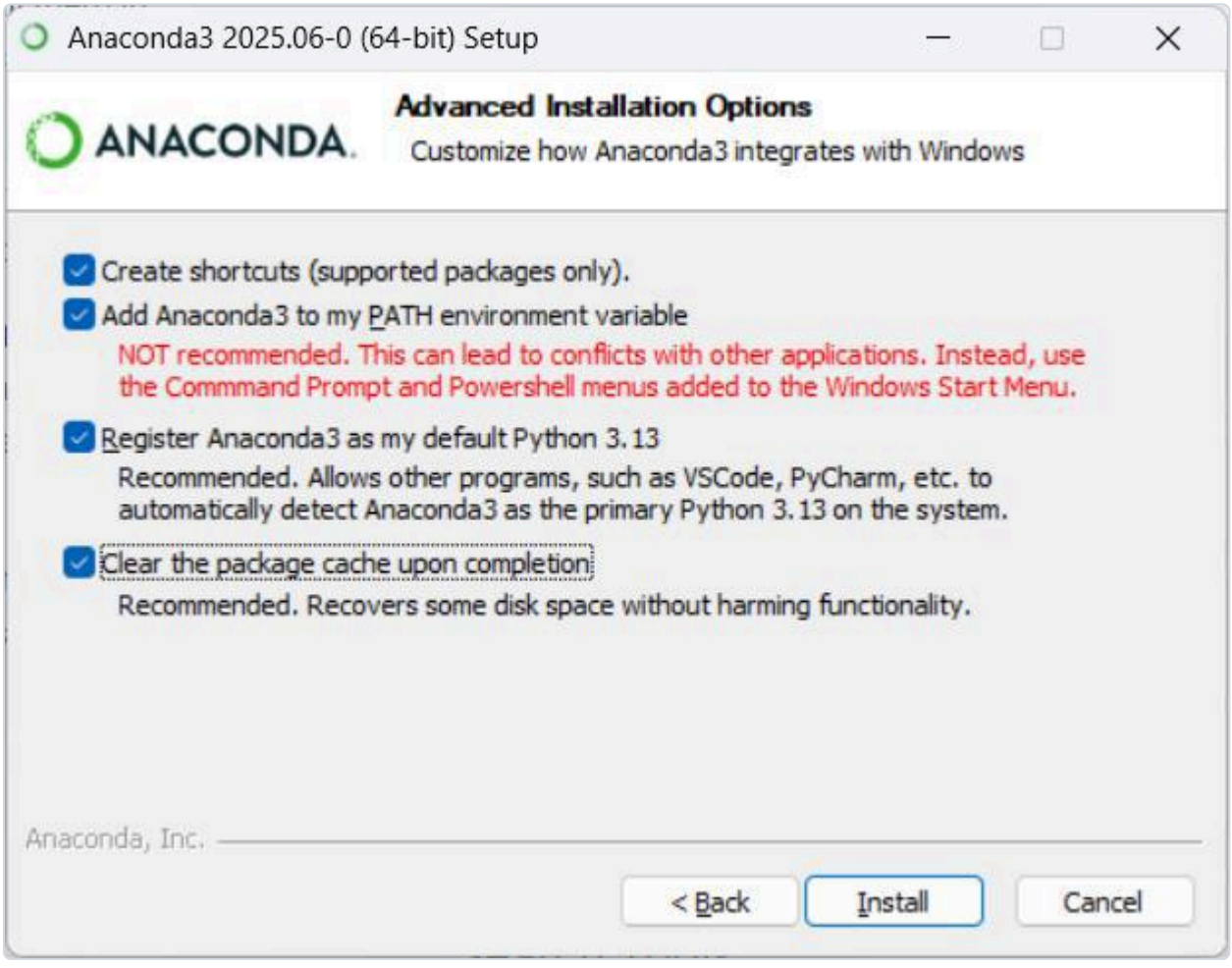
1. **下载（无需注册）**：打开 https://www.anaconda.com/download，选择 **Skip to download** 跳过注册/登录，直接进入下载页。



2. **下载 Windows 安装包**：在下载页 **Distribution Installers** 中选择 Windows 版本（64-bit）。



3. **运行安装程序**：在 **Advanced Options** 勾选第二项 **Add Anaconda3 to my PATH environment variable**（或安装后手动将 Anaconda 目录加入环境变量），方便直接在终端使用 `conda`。



4. **验证**：打开 CMD，输入 `conda`，能正常输出即安装成功。

```
C:\Users\lab_l>conda
usage: conda-script.py [-h] [-v] [--no-plugins] [-V] COMMAND ...

conda is a tool for managing and deploying applications, environments and packages.

options:
  -h, --help            Show this help message and exit.
  -v, --verbose         Can be used multiple times. Once for detailed output, twice for INFO logging, thrice for DEBUG
                        logging, four times for TRACE logging.
  --no-plugins          Disable all plugins that are not built into conda.
  -V, --version         Show the conda version number and exit.

commands:
  The following built-in and plugins subcommands are available.

  COMMAND
    activate            Activate a conda environment.
    build               Build conda packages from a conda recipe.
    clean               Remove unused packages and caches.
    compare             Compare packages between conda environments.
    config              Modify configuration values in .condarc.
    content-trust       Signing and verification tools for Conda
    convert             Convert pure Python packages to other platforms (a.k.a., subdirs).
    create              Create a new conda environment from a list of specified packages.
    deactivate          Deactivate the current active conda environment.
    debug               Debug the build or test phases of conda recipes.
    develop             Install a Python package in 'development mode'. Similar to `pip install --editable`.
```

5. **创建并激活环境**:

`bat conda create -n Dofbot_2026 python=3.10 -y conda activate Dofbot_2026`

> 重开终端时：方案 B 先 `conda activate Dofbot_2026` 再进入项目目录（退出用 `conda deactivate`）。

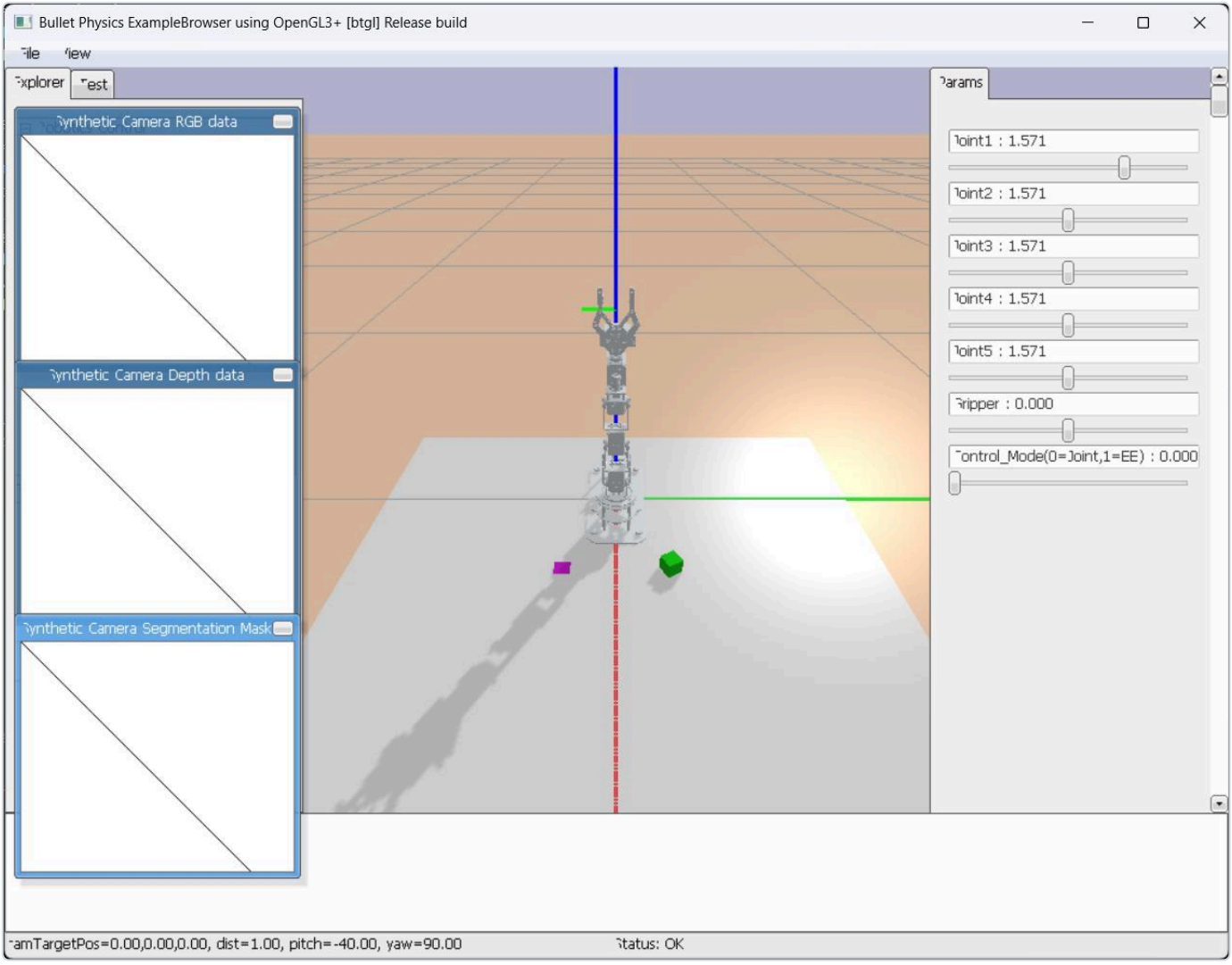
3. 安装依赖并验证

在项目根目录执行（方案 B 先激活环境；方案 A 的 Ubuntu/macOS 将 `python` 换为 `python3.10`）：

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python dofbot_demo.py
```

主要依赖：`roboticstoolbox-python`（机器人学）、`numpy==1.23.5` / `scipy`（数值计算）、`pybullet`（机械臂仿真）、`torch`（FK/IK 学习）、`matplotlib` / `pandas`。NumPy 固定 1.23.5 以避免 PyBullet 的 NumPy 1.x ABI 兼容问题；不要使用 `sudo pip install`。

运行 `python dofbot_demo.py` 会弹出 PyBullet 仿真窗口，能正常弹出即依赖安装成功；关闭窗口后程序结束。



> **不建议** `python -c "import ..."` **测试依赖**：会在同一进程一次性导入多个资源（PyBullet、Torch 等）引发资源竞争，可能误报失败。

4. VS Code 配置

1. 安装 VS Code 与 Microsoft **Python** 扩展（`ms-python.python`）。
2. 用 **文件** → **打开文件夹** 打开项目根目录。
3. `Ctrl/Cmd+Shift+P` → **Python: Select Interpreter**：方案 A 选 Python 3.10，方案 B 选 `Dofbot_2026`（未列出时用 **Enter interpreter path** 手动指定）。
4. 打开 `.py` 文件，用 **Run Python File in Terminal** 运行（带参数时改用终端）。

5. 常用运行命令

前提：已按第 2 节准备好 Python 并进入项目根目录（方案 A 的 Ubuntu/macOS 将 `python` 换为 `python3.10`）。

用途	命令
机械臂交互演示（PyBullet GUI）	`python dofbot_demo.py`
运动学任务	`python dh_kine_student.py`
└ 显示交互图形	`python dh_kine_student.py --show`
└ 调整采样数量	`python dh_kine_student.py --samples 30000`
FK/IK 完整流程	`python dofbot_learn_student.py all --samples 120000 --workers 4 --epochs 200 --batch-size 128`
└ 仅采集数据	`python dofbot_learn_student.py collect --samples 120000 --workers 4`
└ 用已有数据训练	`python dofbot_learn_student.py train --raw "dataset/时间戳/dofbot_fk_120000_raw.csv"`
└ 验证已有模型	`python dofbot_learn_student.py validate --fk-model "results/learn_model/mlp_fk_时间戳/best_model.pt" --ik-model "results/learn_model/mlp_ik_时间戳/best_model.pt" --stats "dataset/时间戳/dofbot_fk_120000_norm_stats.json"`

> 首次验证建议先用较小的 `--samples` 和 `--epochs`（完整流程耗时、占磁盘较大）。`dh_kine_student.py` 部分函数为作业 TODO，出现 `NotImplementedError` 属正常，需先完成作业。

6. 常见问题

pip 下载慢：使用清华镜像。

```
python -m pip install -i https://pypi.tuna.tsinghua.edu.cn/simple -r requirements.txt
```

NumPy 版本冲突：确认已安装版本（`pip show` 只查询信息，不会导入依赖）。

```
python -m pip show numpy
```

项目要求 `numpy==1.23.5`，建议用方案 B（Conda）隔离环境。

PyBullet GUI 打不开：先确认依赖能导入（不要用 `-c` 导入测试）。

```
python dofbot.py
```

若导入正常但 GUI 打不开，更新显卡驱动，并确认不是通过远程桌面或无图形界面环境运行（数据采集使用 `DIRECT` 模式，通常无需 GUI）。

PyTorch 安装慢或体积大：学习代码支持 CPU 运行，不需要 CUDA 时可装 PyTorch CPU 版本，命令按当前 Python 版本参考 PyTorch 官网选择。

找不到 `dofbot` / `utils_learn`：确认当前工作目录是项目根目录（而不是 `utils_learn` 或 `dataset` 子目录），`cd` 到项目根目录后重试。

训练内存不足：减小样本数、批大小或并行进程数。

```
python dofbot_learn_student.py all --samples 20000 --workers 1 --epochs 20 --batch-size 32
```

Ubuntu / macOS 依赖编译报错：先按错误信息确认失败包，确保使用 Python 3.10 及匹配架构，不要取消 `requirements.txt` 的版本限制；若提示缺 C/C++ 工具，安装基础工具：Ubuntu 用 `sudo apt install build-essential`，macOS 用 `xcode-select --install`。